↳ JavaScript is Synchronous.

Date __/__/__
Page _____

Closures

→ "Closure" is a function bind together with its lexical environment.

★ OR

function along with its lexical scope.

Note:- We can assign :-

① function to a variable

e.g.

```
Var a = function y() {
        console.log (a);-
    };
```

② We can pass a function into a function.

e.g.

```
x( function y() {
      Console.log (a);
   });
```

③ Return this function from a function.

e.g.

```
function x() {
  var a = 7;
  function y() {
    console.log(a);
  } return y;
}
var z = x();
```

→ It will return the function itself. & x() will no longer exsist.

→ Not just function was return but a closure is return.

console.log(z); ⟶ this will give function y().

// ------

Suppose there are 1000 lines. then after that if we pass

i.e.

```
f y(){
  console.log(a);
}
```

z(); ⟶ o/p:- 7

---

* **Corner Cases:-**

e.g.

```
function x(){
  var a=7;
  function y(){
  console.log(a);
  }
  a=100;
  return y;
}
var z=x();
console.log(z);
//.....
z();
```

⟶ o/p:- 100

e.g. (2) :—

```
function z(){
  var b = 900;
  function x(){
    var a=7;
    function y(){
    console.log(a, b);
    }
    y();
  }
  x();
}
z();
```

O/p :- 7
900

Here we try to access parent's scope & parent's parent scope also.

* Uses of closures :—

- ↳ Module Design Pattern
- ↳ Currying in JS
- ↳ Functions like once → (It only runs one)
- ↳ Memoize
- ↳ Maintaining state in async world
- ↳ setTimeouts
- ↳ Iterators.

and many more ....